

Fuzzifying Derivatives of Regular Expressions

Women in Logic, FLoC 2026

Mariam Katamashvili David Maisuradze

Kutaisi International University
Vekua Institute of Applied Mathematics, TSU

2026

Outline

- 1 Derivatives
- 2 Automata
- 3 Rocq
- 4 Fuzzy layer
- 5 Conclusion

Where we are

- 1 Derivatives
- 2 Automata
- 3 Rocq
- 4 Fuzzy layer
- 5 Conclusion

Running a regular expression

To test whether a word matches a pattern, we have to *run* the pattern on the word. There is more than one way to do this.

Compile to an automaton

Turn the pattern into a separate machine, then feed the word to it. The classical route: Thompson, Glushkov, the subset construction.

Differentiate the pattern

Never build a machine. The **derivative** is what is left to match after reading a symbol; one derivative per symbol runs the pattern.

We take the second route

And, as we will see, differentiating a pattern hands us the automaton anyway.

The derivative of a regular expression

The derivative $\mathcal{D}_a(r)$ is what remains to be matched once we have read the symbol a . On languages it simply strips a leading a :

$$\partial_a(L) = \{ w \mid a \cdot w \in L \}.$$

On expressions it is computed directly, by recursion on the syntax:

$$\mathcal{D}_a(\emptyset) = \emptyset,$$

$$\mathcal{D}_a(\varepsilon) = \emptyset,$$

$$\mathcal{D}_a(a) = \varepsilon,$$

$$\mathcal{D}_a(b) = \emptyset \ (a \neq b),$$

$$\mathcal{D}_a(r+s) = \mathcal{D}_a(r) + \mathcal{D}_a(s),$$

$$\mathcal{D}_a(r \cdot s) = \mathcal{D}_a(r) \cdot s + \nu(r) \cdot \mathcal{D}_a(s),$$

$$\mathcal{D}_a(r^*) = \mathcal{D}_a(r) \cdot r^*,$$

$$\mathcal{D}_a(r \& s) = \mathcal{D}_a(r) \& \mathcal{D}_a(s),$$

$$\mathcal{D}_a(\neg r) = \neg \mathcal{D}_a(r).$$

Example

$$\mathcal{D}_a(ab + ac) = \mathcal{D}_a(ab) + \mathcal{D}_a(ac) = b + c.$$

Derivatives with respect to a word

Iterating symbol by symbol gives the derivative with respect to a whole word, the same way on languages and on expressions:

$$\partial_\varepsilon(L) = L, \quad \partial_{au}(L) = \partial_u(\partial_a(L)); \quad \mathcal{D}_\varepsilon(r) = r, \quad \mathcal{D}_{au}(r) = \mathcal{D}_u(\mathcal{D}_a(r)).$$

Example

$\mathcal{D}_{ab}(ab + ac) = \mathcal{D}_b(\mathcal{D}_a(ab + ac)) = \mathcal{D}_b(b + c) = \varepsilon$, which is exactly the statement that the word ab belongs to $\mathcal{L}(ab + ac)$.

Theorem (Semantic-syntactic correspondence)

Differentiating the expression and differentiating the language it denotes give the same language: $\mathcal{L}(\mathcal{D}_u(r)) = \partial_u(\mathcal{L}(r))$.

Theorem (Regularity)

If $L \subseteq \Sigma^$ is regular, then $\partial_u(L)$ is also regular for any word $u \in \Sigma^*$.*

Theorem (Finiteness)

Let Σ be a finite alphabet and let $L \subseteq \Sigma^$ be a regular language. Then the set of all derivatives of L with respect to words over Σ ,*

$$\{ \partial_u(L) \mid u \in \Sigma^* \},$$

is finite.

Canonicalization

To turn “finitely many derivatives” into a genuinely finite set, we rewrite every expression to a single canonical form $[r] = \text{nf}(r)$, chosen so that any two expressions the standard laws identify collapse to the same syntax tree. Then $r \sim s$ exactly when $[r] = [s]$.

Each operator has its own rules:

- $+$: flatten, drop $\emptyset$, sort, remove duplicates.
- $\cdot$: flatten; collapse to $\emptyset$ if a factor is $\emptyset$, otherwise drop ε .
- $*$: $\emptyset^* = \varepsilon$, $\varepsilon^* = \varepsilon$, $(r^*)^* = r^*$.

Example

$$((a^*)^* \cdot \varepsilon) + (b + (a + \emptyset)) + b$$

normalizes to

$$a + b + a^*.$$

Where we are

- 1 Derivatives
- 2 Automata**
- 3 Rocq
- 4 Fuzzy layer
- 5 Conclusion

From derivatives to automata

Since reading a symbol is the same as taking a derivative, and since a regular expression has only finitely many derivatives once we keep them canonical, we can take those canonical derivatives to be the states of a deterministic automaton.

Definition (Derivative DFA)

$$Q = \{[\mathcal{D}_u(r)]\}, \quad q_0 = [r], \quad \delta(q, a) = [\mathcal{D}_a(q)], \quad F = \{q : \nu(q) = \varepsilon\}.$$

Theorem (Correctness)

The automaton accepts exactly the language of the expression: $\mathcal{L}(M) = \mathcal{L}(r)$.

If we could quotient by full semantic equivalence we would get the *minimal* DFA. Canonical forms give a finite and very often minimal automaton, but not always.

Constructing the automaton

We build the automaton by exploring outward from the start state: at each state we differentiate, canonicalize, and add every new state we reach, stopping when nothing new appears.

The search is depth-first. A canonical derivative already in Q only records a transition and ends the branch. One not yet in Q is added and explored recursively.

```
dfa(r):  
    q0      := [r]          (* canonical start *)  
    Q       := { q0 }  
    delta := empty  
    for each a in Sigma do  
        goto(q0, a)  
    F := { q in Q | nullable q }  
  
goto(q, a):  
    qa := [ D_a(q) ]      (* derive + canon *)  
    if qa in Q then  
        delta[q,a] := qa  (* already seen *)  
    else  
        Q          += qa  (* new state *)  
        delta[q,a] := qa  
        for each a' in Sigma do  
            goto(qa, a')  (* recurse *)
```

Example: the automaton of $s^* = (a+m+r.i)^*$



Initialize.

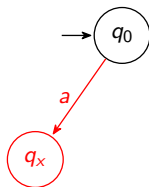
Write $s = a+m+r.i$. The start state is the canonical form of the regex s^* ,

$$q_0 = [s^*].$$

Known states: $\{q_0\}$. Now call goto on q_0 for each symbol a, m, r, i .

	a	m	r	i	acc.
q_0					

Example: the automaton of $s^* = (a+m+r \cdot i)^*$



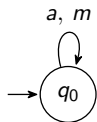
goto(q_0, a)

$$\begin{aligned}\mathcal{D}_a(s^*) &= \mathcal{D}_a(s) \cdot s^* \\ &= (\mathcal{D}_a(a) + \mathcal{D}_a(m) + \mathcal{D}_a(r) \cdot i) \cdot s^* \\ &= (\varepsilon + \emptyset + \emptyset \cdot i) \cdot s^*\end{aligned}$$

Left as is, this messy expression is a **brand new state** q_x (in red). But if we canonicalize it, it folds away.

	a	m	r	i	acc.
q_0					

Example: the automaton of $s^* = (a+m+r \cdot i)^*$



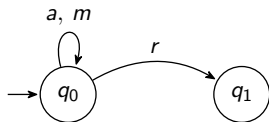
Canonicalize

$$(\varepsilon + \emptyset + \emptyset \cdot i) \cdot s^* \sim \varepsilon \cdot s^* \sim s^* = q_0$$

It **collapses into** q_0 . Record the self-loop $q_0 \xrightarrow{a} q_0$; reading m is identical, so $q_0 \xrightarrow{m} q_0$ too. Without canonicalization each such form would be a fresh state and the automaton **would never stop growing**.

	a	m	r	i	acc.
q_0	q_0	q_0			

Example: the automaton of $s^* = (a+m+r \cdot i)^*$



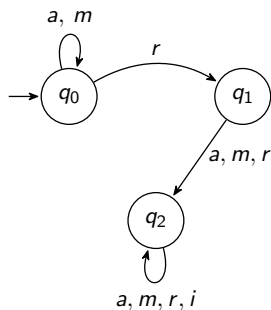
goto(q_0, r)

$$\begin{aligned}
 \mathcal{D}_r(s^*) &= \mathcal{D}_r(s) \cdot s^* \\
 &= (\emptyset + \emptyset + \mathcal{D}_r(r) \cdot i) \cdot s^* \\
 &= (\varepsilon \cdot i) \cdot s^* \sim i \cdot s^*
 \end{aligned}$$

A **new** canonical form. Add it as $q_1 = i \cdot s^*$, record $q_0 \xrightarrow{r} q_1$, and **recurse into** q_1 .

	a	m	r	i	acc.
q_0	q_0	q_0	q_1		
q_1					

Example: the automaton of $s^* = (a+m+r.i)^*$



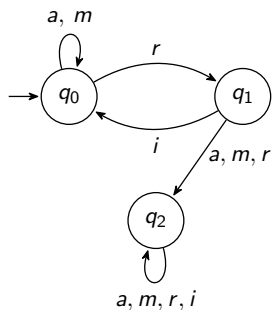
$\text{goto}(q_1, a), (q_1, m), (q_1, r), \quad q_1 = i \cdot s^*$

$$\begin{aligned} \mathcal{D}_x(i \cdot s^*) &= \mathcal{D}_x(i) \cdot s^* + \nu(i) \cdot \mathcal{D}_x(s^*) \\ &= \mathcal{D}_x(i) \cdot s^* = \emptyset \cdot s^* \sim \emptyset \quad (x \in \{a, m, r\}) \end{aligned}$$

A **new** canonical form $\emptyset$, the **dead state** q_2 . Add it, record $q_1 \xrightarrow{a, m, r} q_2$, and recurse: $\mathcal{D}_x(\emptyset) = \emptyset$, so q_2 loops to itself on every symbol.

	a	m	r	i	acc.
q_0	q_0	q_0	q_1		
q_1	q_2	q_2	q_2		
q_2	q_2	q_2	q_2	q_2	

Example: the automaton of $s^* = (a+m+r.i)^*$



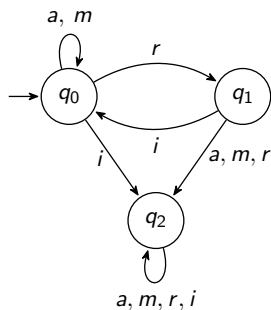
goto(q_1, i)

$$\mathcal{D}_i(i \cdot s^*) = \mathcal{D}_i(i) \cdot s^* = \varepsilon \cdot s^* \sim s^* = q_0$$

Already known. Record $q_1 \xrightarrow{i} q_0$ and return. q_1 is done, so we climb back up to q_0 .

	a	m	r	i	acc.
q_0	q_0	q_0	q_1		
q_1	q_2	q_2	q_2	q_0	
q_2	q_2	q_2	q_2	q_2	

Example: the automaton of $s^* = (a+m+r.i)^*$



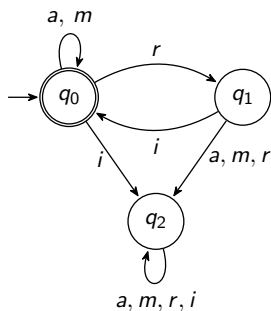
$\text{goto}(q_0, i)$

$$\begin{aligned}\mathcal{D}_i(s^*) &= \mathcal{D}_i(s) \cdot s^* \\ &= (\emptyset + \emptyset + \emptyset \cdot i) \cdot s^* \sim \emptyset = q_2\end{aligned}$$

Already known: the dead state. Record $q_0 \xrightarrow{i} q_2$. Reading i with no r before it fails.

	a	m	r	i	acc.
q_0	q_0	q_0	q_1	q_2	
q_1	q_2	q_2	q_2	q_0	
q_2	q_2	q_2	q_2	q_2	

Example: the automaton of $s^* = (a+m+r \cdot i)^*$



	a	m	r	i	acc.
q_0	q_0	q_0	q_1	q_2	yes
q_1	q_2	q_2	q_2	q_0	no
q_2	q_2	q_2	q_2	q_2	no

Accepting states and halt

$$q_0 = s^*, \quad q_1 = i \cdot s^*, \quad q_2 = \emptyset.$$

$$\nu(q_0) = \varepsilon \Rightarrow \text{yes}, \quad \nu(q_1) = \emptyset \Rightarrow \text{no}, \quad \nu(q_2) = \emptyset \Rightarrow \text{no}$$

No new canonical form appears, so the construction **halts**. From $s^* = (a+m+r \cdot i)^*$ we have built a three-state DFA whose words are sequences of the blocks a , m and ri ; for instance it accepts *mariam*. The dead state q_2 catches an i with no r before it.

Where we are

- 1 Derivatives
- 2 Automata
- 3 Rocq**
- 4 Fuzzy layer
- 5 Conclusion

Why formalize, and what is Rocq?

The derivative theory is easy to state and easy to get subtly wrong: the concatenation case, the star case, the finiteness argument. “This case is trivial” and “this follows directly” are exactly where mistakes hide. So we machine-check it.

Rocq (formerly Coq) is an interactive proof assistant: we state each theorem and build a proof that a small, **trusted kernel** verifies mechanically, with no step left to take on trust.

Used in practice

The same tool has checked the CompCert verified C compiler and Gonthier’s proof of the Four Colour Theorem.

Rocq is built on a **dependent type theory**, the Calculus of Inductive Constructions - so its very foundation is a piece of logic.

- By the **Curry-Howard correspondence**, a proposition is a type and a proof is a term of that type, so proving a statement means constructing such a term.
- Because the logic is **constructive**, every proof carries computational content: a proof that something exists hands you an actual witness you can compute with, not merely an assurance that one exists.

What a proof really is

We prove the semantic-syntactic correspondence, $w \in \mathcal{D}_u(r) \iff u++w \in r$. This is the script we write:

```
Lemma D_correct : forall u r w,  
  (w \in D u r) = ((u ++ w) \in r).
```

Proof.

```
  elim=> [|a u IHu] r w //=.  
  rewrite IHu D_char_correct.  
  by rewrite -cat_cons.
```

Qed.

Three tactic lines. They elaborate to the term the kernel actually checks, on the next slide.

The term the kernel checks

What Print D_correct returns – no tactics remain:

```
D_correct =
fun _top_assumption_ : word =>
(fun _eval_0_ : (fun l : seq A =>
  forall (r : regex) (w : word), (w \in D l r) = (l ++ w \in r)) [::] =>
(list_ind
  (fun l : seq A =>
    forall (r : regex) (w : word), (w \in D l r) = (l ++ w \in r))
  _eval_0_)^~ _top_assumption_)
(fun (r : regex) (w : word) => erefl)
(fun (a : A) (u : seq A)
  (IHu : forall (r : regex) (w : word), (w \in D u r) = (u ++ w \in r))
  (r : regex) (w : word) =>
(fun _eval_0_ : (u ++ w \in D_char a r) = (a :: u ++ w \in r) =>
  eq_ind_r (eq~ (a :: u ++ w \in r)) _eval_0_ (IHu (D_char a r) w))
  ((fun _eval_0_ : (a :: u ++ w \in r) = (a :: u ++ w \in r) =>
    eq_ind_r (eq~ (a :: u ++ w \in r)) _eval_0_
      (D_char_correct a r (u ++ w)))
    ((fun _eval_0_ : ((a :: u) ++ w \in r) = (a :: u ++ w \in r) =>
      eq_ind ((a :: u) ++ w)
        (fun s : word => (s \in r) = (a :: u ++ w \in r))
        _eval_0_ (a :: u ++ w) (cat_cons a u w))
      erefl))))
```


Where we are

- 1 Derivatives
- 2 Automata
- 3 Rocq
- 4 Fuzzy layer**
- 5 Conclusion

When exact matching is too strict

Everything so far rests on one yes-or-no question: are two symbols the *very same* symbol? That is often too strict. Real text is noisy - symbols get mistyped or swapped for lookalikes, OCR and speech hand us each symbol with only a confidence, and in biological sequences chemically close bases should count as near matches.

We would like a pattern to keep matching when the input is merely *close* to it, and derivatives to do the same.

The question

What happens if we keep the crisp derivative theory exactly as it is and change one thing – matching two symbols now means matching *up to μ -similarity*?

Two ways to fuzzify

Fuzzifying regular languages is not new, and there are two quite different ways to go about it.

The first way puts the fuzziness on **membership degrees**: a word no longer simply belongs to a language, it belongs to a degree in $[0, 1]$. Some approaches attach a scalar to a regular expression, so that membership in its language is scaled by that value.

The second way puts it on **similarity**: we give the symbols a similarity relation and extend it to words and languages.

Membership stays crisp; what softens is the equality underneath it.

Why similarity

Closeness is a property of the data - the letters, the bases, the sounds - not a number we glue onto the language afterwards. Similarity simply replaces the exact equality of symbols with a graded notion of closeness.

Similarity relations on symbols

A fuzzy relation grades how close two symbols are, $\mathcal{R} : \Sigma \times \Sigma \rightarrow [0, 1]$. We ask for two levels of structure.

Definition (Proximity and similarity)

$\mathcal{R}$ is a *proximity* relation if it is reflexive, $\mathcal{R}(a, a) = 1$, and symmetric, $\mathcal{R}(a, b) = \mathcal{R}(b, a)$. It is a *similarity* relation if it also satisfies the reverse triangle inequality for a T-norm $\otimes$:

$$\mathcal{R}(a, c) \geq \mathcal{R}(a, b) \otimes \mathcal{R}(b, c).$$

A **T-norm** $\otimes$ generalizes conjunction to $[0, 1]$ - commutative, associative, monotone, with unit 1; throughout we take Gödel's minimum, $\otimes = \min$. A **cut** $\mu \in (0, 1]$ turns degrees back into yes-or-no through the crisp relation $\mathcal{R}_\mu = \{(a, b) \mid \mathcal{R}(a, b) \geq \mu\}$.

Example

$\mathcal{R}(a, b) = 0.9$, $\mathcal{R}(a, c) = 0.2$. With cut $\mu = 0.8$: $a \sim_\mu b$ since $0.9 \geq 0.8$, but $a \not\sim_\mu c$.

From symbols to words and languages

The similarity on symbols extends, letter by letter, to words, and then to whole languages.

Definition (Similarity on words)

For $w_1 = a_1 \cdots a_n$ and $w_2 = b_1 \cdots b_n$ of equal length, $\mathcal{R}^w(w_1, w_2) = \bigotimes_{1 \leq i \leq n} \mathcal{R}(a_i, b_i)$; for words of different length $\mathcal{R}^w(w_1, w_2) = 0$.

Definition (Similarity on languages)

$$\mathcal{R}^L(L_1, L_2) = \min \left\{ \inf_{w_1 \in L_1} \sup_{w_2 \in L_2} \mathcal{R}^w(w_1, w_2), \inf_{w_2 \in L_2} \sup_{w_1 \in L_1} \mathcal{R}^w(w_1, w_2) \right\}.$$

Both $\mathcal{R}^w$ and $\mathcal{R}^L$ are again similarity relations.

Similarity on expressions

Similarity also extends to expressions, comparing two of the *same shape* by the similarity of the symbols in corresponding positions.

Definition (Similarity on expressions)

$$\mathcal{R}^E(r_1, r_2) = \begin{cases} \bigotimes_{1 \leq i \leq n} \mathcal{R}(a_i, b_i) & \text{if } r_1 = r[a_1, \dots, a_n] \text{ and } r_2 = r[b_1, \dots, b_n]; \\ 0 & \text{otherwise.} \end{cases}$$

This makes $\mathcal{R}^E$ *syntax-sensitive*: $a + b$ and $b + a$ denote the same language, yet may have $\mathcal{R}^E(a+b, b+a) = 0$.

Shape bounds meaning

Syntactic similarity is always a lower bound on semantic similarity, and the semantic value is what you recover by ranging over all expressions with the right languages.

Theorem (Bridge between $\mathcal{R}^E$ and $\mathcal{R}^L$)

For all restricted regular expressions r_1, r_2 ,

$$\mathcal{R}^E(r_1, r_2) \leq \mathcal{R}^L(L(r_1), L(r_2)),$$

$$\mathcal{R}^L(L(r_1), L(r_2)) = \sup_{\substack{L(r_1)=L(r'_1) \\ L(r_2)=L(r'_2)}} \mathcal{R}^E(r'_1, r'_2).$$

Example

$$\mathcal{R}^E(a+b, b+a) = 0, \quad \mathcal{R}^L(L(a+b), L(b+a)) = 1: \quad 0 \leq 1.$$

Intersection and complement break this bridge, so the fuzzy theorems live on the **restricted** fragment.

Why the restricted fragment?

A **restricted** regular expression uses only union, concatenation, and star - no intersection ($\wedge$) or complement ($\neg$).

Counterexample

Let $\mathcal{R}(a, b) = 0.3 > 0$ for distinct symbols a, b . Take $r_1 = a \wedge a$ and $r_2 = a \wedge b$.

- Semantically: $L(r_1) = \{a\}$, $L(r_2) = \{a\} \cap \{b\} = \emptyset$, so $\mathcal{R}^L(L(r_1), L(r_2)) = 0$.
- Syntactically, comparing piece by piece: $\mathcal{R}^E(r_1, r_2) = 1 \otimes \mathcal{R}(a, b) = 0.3$.

So $\mathcal{R}^E(r_1, r_2) = 0.3 > 0 = \mathcal{R}^L(L(r_1), L(r_2))$: syntax is no longer a lower bound once intersection is allowed - hence the restriction.

Fuzzy derivatives

Now we change matching inside the derivative itself: a symbol counts as read when it is μ -similar to the input, not only when it is equal.

Definition (Fuzzy language derivative)

$$\partial_a^\mu(L) = \{ w \mid \exists b \in \Sigma, \exists v \in \Sigma^* : \mathcal{R}(a, b) \otimes \mathcal{R}^w(w, v) \geq \mu, bv \in L \}.$$

Definition (Fuzzy expression derivative)

$$\mathcal{D}_a^\mu(r) = \sum_{\mathcal{R}(a, b) \otimes \mathcal{R}^E(s, r) \geq \mu} \mathcal{D}_b(s),$$

a *finite* sum of ordinary Brzozowski derivatives, since only finitely many s satisfy $\mathcal{R}^E(s, r) \neq 0$ when Σ is finite.

Theorem (Regularity is preserved)

If L is regular then $\partial_a^\mu(L)$ is regular; if r is a restricted expression then $\mathcal{D}_a^\mu(r)$ is again a restricted expression.

Gödel's min: fuzzy = crisp over a quotient

With Gödel's minimum the cut relation $a \sim_{\mu}^{\Sigma} b \iff \mathcal{R}(a, b) \geq \mu$ is not only reflexive and symmetric but **transitive**, so it is an *equivalence*. We may then bundle μ -similar symbols into classes.

Definition (Quotient alphabet)

$\Sigma^{\mu} = \Sigma / \sim_{\mu}^{\Sigma}$, with quotient map $q_{\mu}(a) = [a]_{\mu}$, extended to words by q_{μ}^* , and image $L^{\mu} = q_{\mu}^*(L)$.

Theorem (Reduction to the quotient alphabet)

A fuzzy derivative over Σ is exactly an ordinary derivative over the quotient alphabet Σ^{μ} :

$$\partial_a^{\mu}(L) = \{ w \mid q_{\mu}^*(w) \in \partial_{[a]_{\mu}}(L^{\mu}) \}, \quad q_{\mu}^*(\partial_a^{\mu}(L)) = \partial_{[a]_{\mu}}(L^{\mu}).$$

Everything carries over

Because the fuzzy theory is just the crisp theory read over Σ^μ , each earlier result transfers.

- **Fuzzy interpretation:** $\mathcal{L}^\mu(r) = \{ w \mid w \sim_\mu^{\Sigma^*} v \text{ for some } v \in L(r) \}$.
- **Correspondence:** $\mathcal{L}^\mu(\mathcal{D}_u^\mu(r)) = \partial_u^\mu(L(r))$.
- **Finiteness:** $\{ \partial_u^\mu(L) \mid u \in \Sigma^* \}$ is finite.
- **Fuzzy automaton:** $\mathcal{L}(M^\mu) = \mathcal{L}_\Sigma^\mu(r)$, the words μ -close to some word of r .

And the quotient transition reads back as a **fuzzy transition**: $\delta(q_1, [a]) = q_2$ holds with membership degree μ .

Where we are

- 1 Derivatives
- 2 Automata
- 3 Rocq
- 4 Fuzzy layer
- 5 Conclusion**

What we have shown.

- The classical derivative theory, up to DFA correctness, is formalized in Rocq.
- We fuzzify through **similarity on symbols** rather than membership degrees.
- The **quotient alphabet** Σ^μ reduces the fuzzy theory to the crisp one, so every earlier result carries over unchanged.

Future work.

- Formalize the **fuzzy layer** in Rocq.
- **Constructive finiteness**: a Kuratowski-finiteness result via normalized representatives.
- Generalize beyond Gödel's min to a **general T-norm**.

Thank you

Questions?